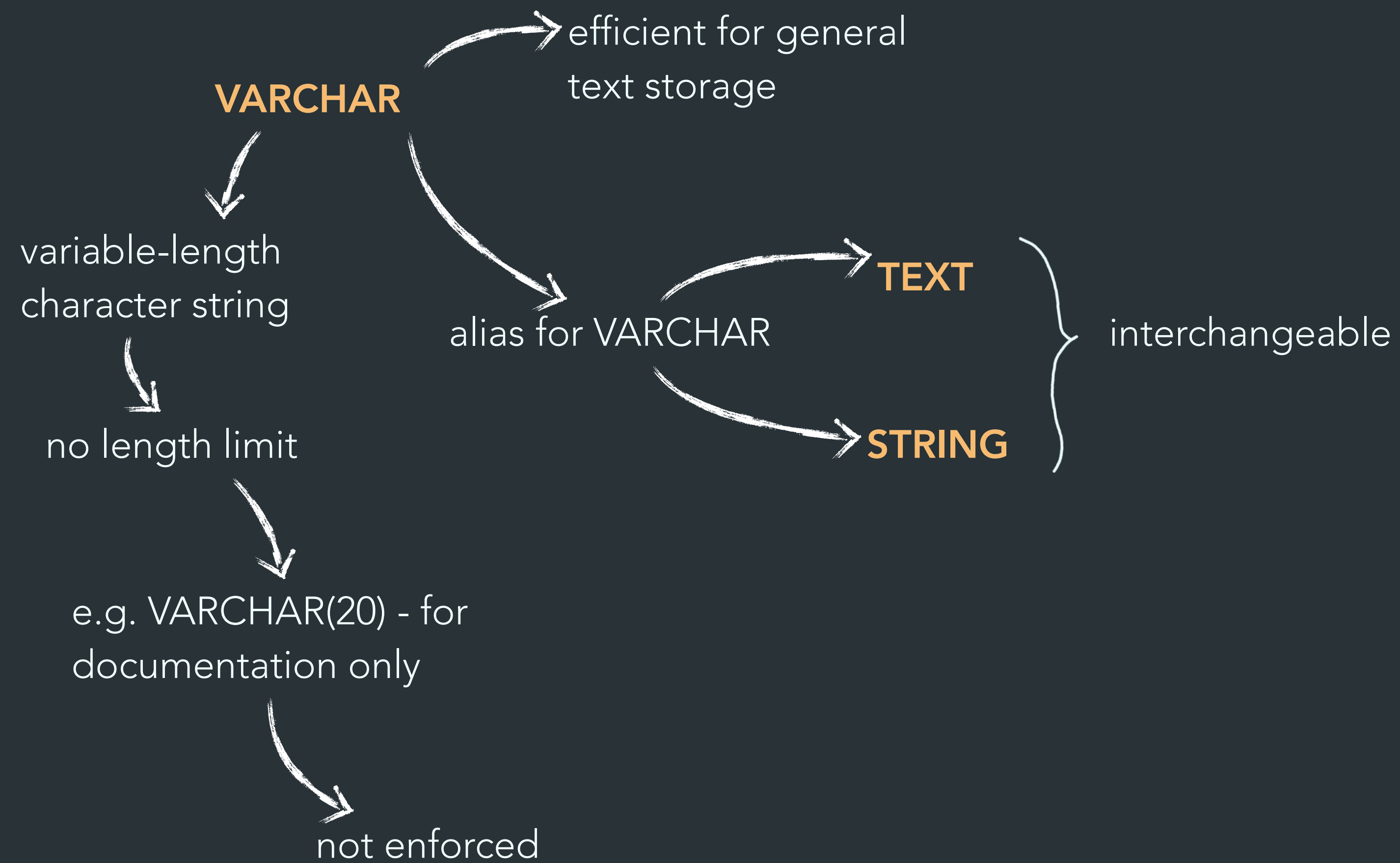




kokchun giang

working with
strings in sql and
duckdb to format
and clean text

text **data types** and their distinctions in duckdb



using array-like **slicing and indexing** to get substrings

SELECT

```
sql_word,  
sql_word[:2],  
sql_word[2:5]
```

FROM

```
staging.sql_glossary;
```

slices from
index 1 to 2
(SQL counts
from 1)

slices from index 2 to 5,
so gets the substring
composing of characters
on index 2 to 5

SELECT

```
sql_word,  
sql_word[0],  
sql_word[1],  
sql_word[-1],
```

FROM

```
staging.sql_glossary;
```

returns empty string
as there is no strings
in 0th position

gets the last
character in the
string

using **string functions** to transform text

```
SELECT
    upper(trim(sql_word, ' ')) as trimmed_word,
    trimmed_word[1],
    trimmed_word[-1]
FROM
    staging.sql_glossary;
```

composition of functions or
nested functions

many useful **text functions** in the documentation

Text Functions and Operators

This section describes functions and operators for examining and manipulating `STRING` values.

Name	Description
<code>string ^@ search_string</code>	Return true if <code>string</code> begins with <code>search_string</code> .
<code>string string</code>	Concatenate two strings. Any NULL input results in NULL. See also <code>concat(string,...)</code> .
<code>string[index]</code>	Extract a single character using a (1-based) index.
<code>string[begin:end]</code>	Extract a string using slice conventions, see slicing .
<code>string LIKE target</code>	Returns true if the <code>string</code> matches the like specifier (see Pattern Matching).
<code>string SIMILAR TO regex</code>	Returns <code>true</code> if the <code>string</code> matches the <code>regex</code> ; identical to <code>regexp_full_match</code> (see Pattern Matching).
<code>array_extract(list, index)</code>	Extract a single character using a (1-based) index.
<code>array_slice(list, begin, end)</code>	Extract a string using slice conventions. Negative values are accepted.
<code>ascii(string)</code>	Returns an integer that represents the Unicode code point of the first character of the <code>string</code> .
<code>bar(x, min, max[, width])</code>	Draw a band whose width is proportional to <code>(x - min)</code> and equal to <code>width</code> characters when <code>x</code> is

using **regex** for more complex pattern matching

```
SELECT
    regexp_replace (trim(description), ' +', ' ', 'g') AS cleaned_description,
    regexp_replace (trim(lower(example)), ' +', ' ', 'g') AS example,
FROM
    staging.sql_glossary;
```

replaces every matching of
the pattern ' +' (one or more
spaces) and replaces it
with ' ' (one space)

using schemas to divide into **layers** for structure

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE TABLE IF NOT EXISTS staging.sql_glossary AS (  
    SELECT * FROM read_csv_auto('data/sql_terms.csv')  
);
```

staging schema is used to
represent a staging layer,
kind of landing zone for
data

```
CREATE SCHEMA IF NOT EXISTS refined;
```

```
CREATE TABLE  
    IF NOT EXISTS refined.sql_glossary AS (  
        SELECT  
            UPPER(TRIM(sql_word)) AS sql_word,  
            description,  
            example  
        FROM  
            staging.sql_glossary  
    );
```

refined layer or
warehouse layer is where
data is transformed